

```

// C++ program to print DFS traversal from
#include<iostream>
#include<list>
using namespace std;

// Graph class represents a directed graph using adjacency list representation
class Graph
{
    int V; // No. of vertices
    list<int> *adj; // Pointer to an array containing adjacency lists
    void DFSUtil(int v, bool visited[]); // A recursive function used by DFS

public:
    Graph(int V); // Constructor
    void addEdge(int v, int w); // function to add an edge to graph
    void DFS(int v); // DFS traversal of the vertices reachable from v
};

Graph::Graph(int V)
{
    this->V = V;
    adj = new list<int>[V];
}

void Graph::addEdge(int v, int w)
{
    adj[v].push_back(w); // Add w to v's list.
}

void Graph::DFSUtil(int v, bool visited[])
{
    // Mark the current node as visited and print it
    visited[v] = true;
    cout << v << " ";

    // Recur for all the vertices adjacent to this vertex
    list<int>::iterator i;
    for (i = adj[v].begin(); i != adj[v].end(); ++i)
        if (!visited[*i])
            DFSUtil(*i, visited);
}

// DFS traversal of the vertices reachable from v. It uses recursive DFSUtil()
void Graph::DFS(int v)
{
    bool *visited = new bool[V]; // Mark all the vertices as not visited
    for (int i = 0; i < V; i++)
        visited[i] = false;

    DFSUtil(v, visited); // Call the recursive helper function to print DFS traversal
}

```

```
int main()
{
    Graph g(4); // Create a graph given in the above diagram
    g.addEdge(0, 1);
    g.addEdge(0, 2);
    g.addEdge(1, 2);
    g.addEdge(2, 0);
    g.addEdge(2, 3);
    g.addEdge(3, 3);

    cout << "Following is Depth First Traversal"
          " (starting from vertex 2) \n";
    g.DFS(2);

    return 0;
}
```